KENNESAW STATE UNIVERSITY

CS 4277

Variational Autoencoder

Dr. Chen Zhao

czhao4@kennesaw.edu

Computer Science

College of Computing and Software Engineering

Learning Objectives

- Introduction
- Autoregressive Models
- From AE to VAE
- ELOB and VAE
- Application of VAE

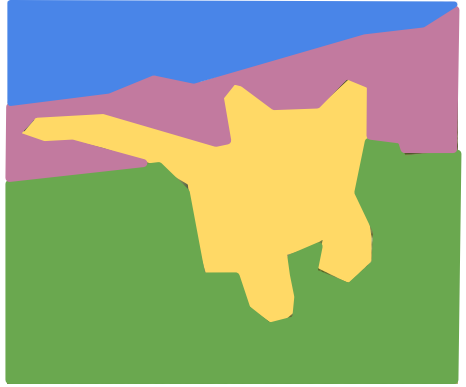
Introduction

Supervised Learning

- Supervised Learning
- Data: (x, y)
- x is data, y is label
- Goal: Learn a function to map x -> y
- Examples: Classification, regression, object detection, semantic segmentation, image captioning, etc.

Cat

Classification



GRASS, CAT, TREE, SKY

Semantic Segmentation

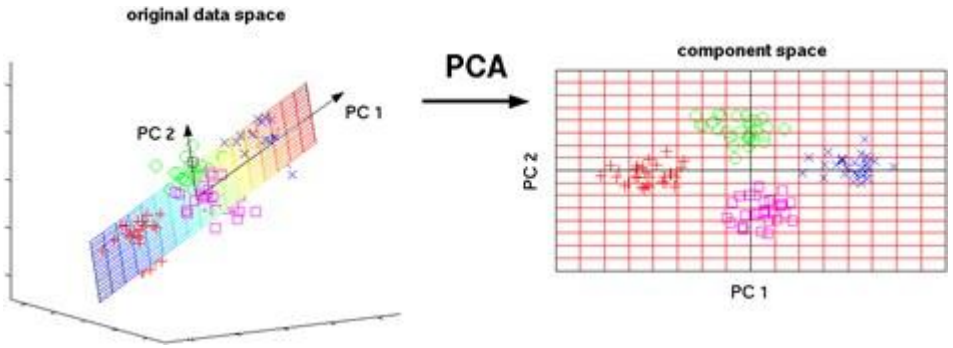
A cat sitting on a suitcase on the floor

Image captioning

DOG, DOG, CAT

Object Detection

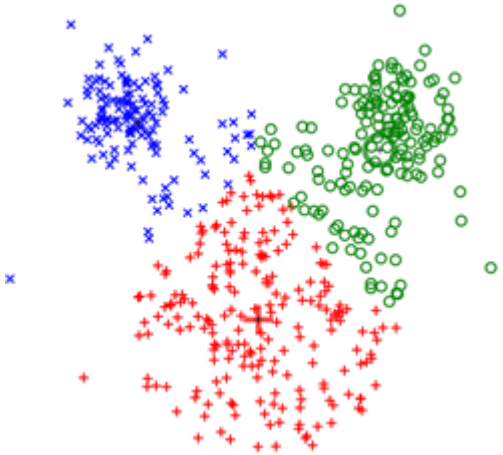
Unsupervised Learning



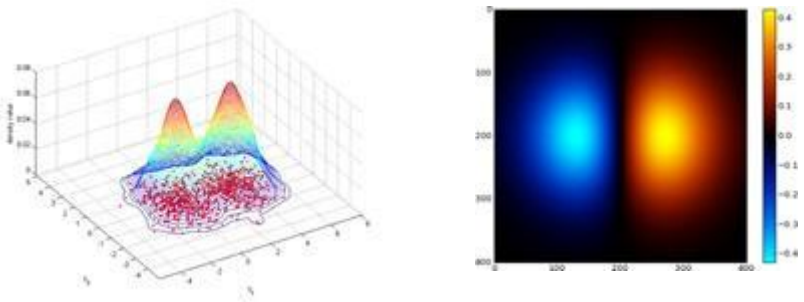
3-d → 2-d

Principal Component Analysis
(Dimensionality reduction)

- Unsupervised Learning
- Data: x
- Just data, no labels!
- Goal: Learn hidden structure in data
- Examples: Clustering, dimensionality reduction, density estimation, etc.



K-means clustering



2-d density estimation

Modeling p(x)

Generative vs Discriminative Models

- **Discriminative Model**: Learn a probability distribution $p(y|x)$
- **Generative Model**: Learn a probability distribution $p(x)$
- **Conditional Generative Model**: Learn $p(x|y)$

Data: x

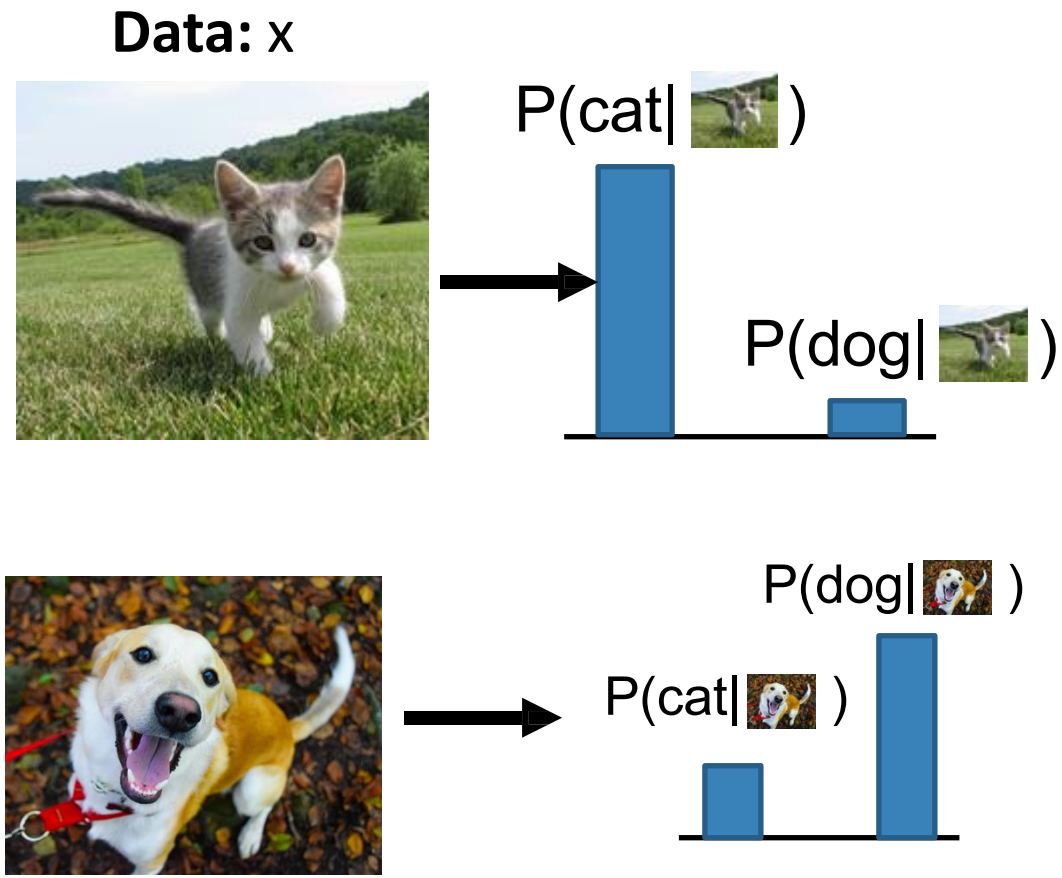
Label: y
Cat

Probability Recap:

- Density Function: $p(x)$ assigns a positive number to each possible x; higher numbers mean x is more likely.
- Density functions are normalized:

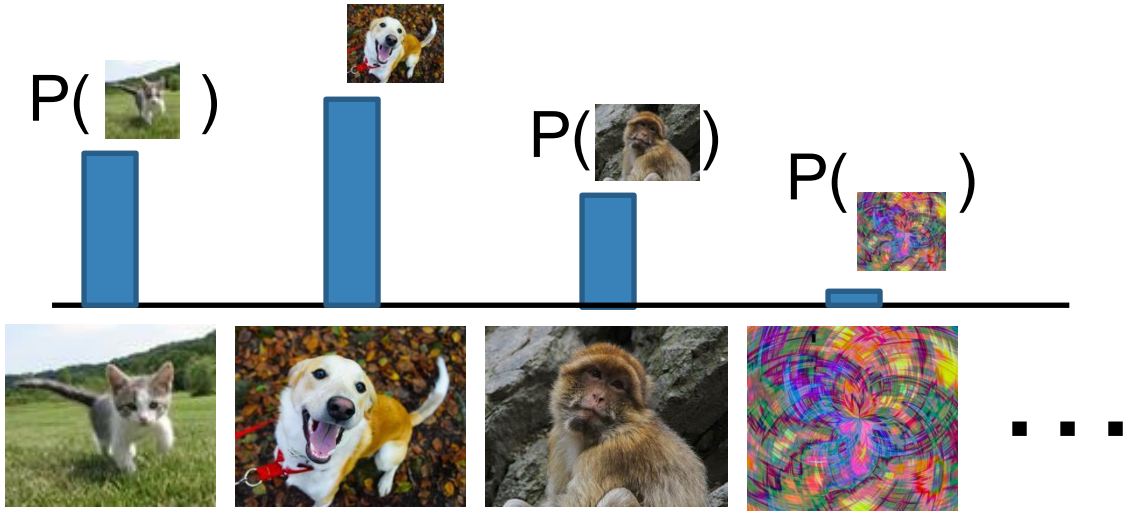
$$\int_x p(x)dx = 1$$

- Different values of x compete for density



Generative vs Discriminative Models

- **Generative Model**: Learn a probability distribution $p(x)$
- Requires deep understanding: Is a dog more likely to sit or stand? Is a 3-legged dog more likely than a 3-armed monkey?
- Model can “reject” unreasonable inputs by giving them small probability mass



All possible **images** compete for probability mass

Recall Bayes' Rule:

- Bayes' rule:

$$p(x|y) = \frac{p(y|x)}{p(y)} p(x)$$

- Discriminative model: $p(y|x)$
- Conditional generative model: $p(x|y)$
- Prior over labels: $p(y)$
- (Unconditional) Generative Models: $p(x)$
- We can build a conditional generative model from other components … but not common in practice

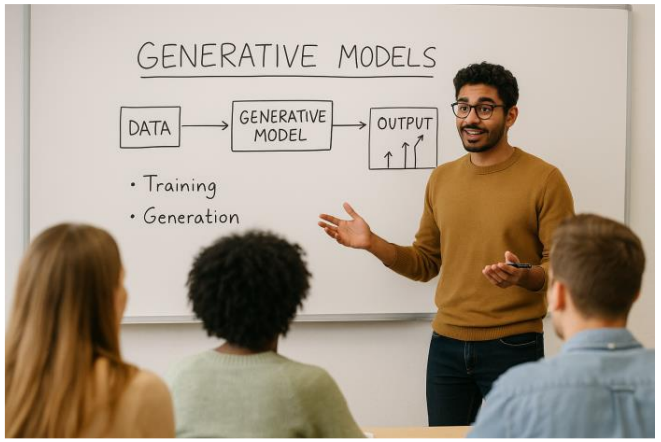
Generative vs Discriminative Models

- **Discriminative Model**: Learn a probability distribution $p(y|x)$
  - Assign labels to data Feature learning (with labels)
- **Generative Model**: Learn a probability distribution $p(x)$
  - Detect outliers,
  - Feature learning (without labels),
  - Sample to generate new data
- **Conditional Generative Model**: Learn $p(x|y)$
  - Assign labels while rejecting outliers
  - Sample to generate data from labels
- Generative models" means either of GM and CGM; conditional generative models are most common in practice

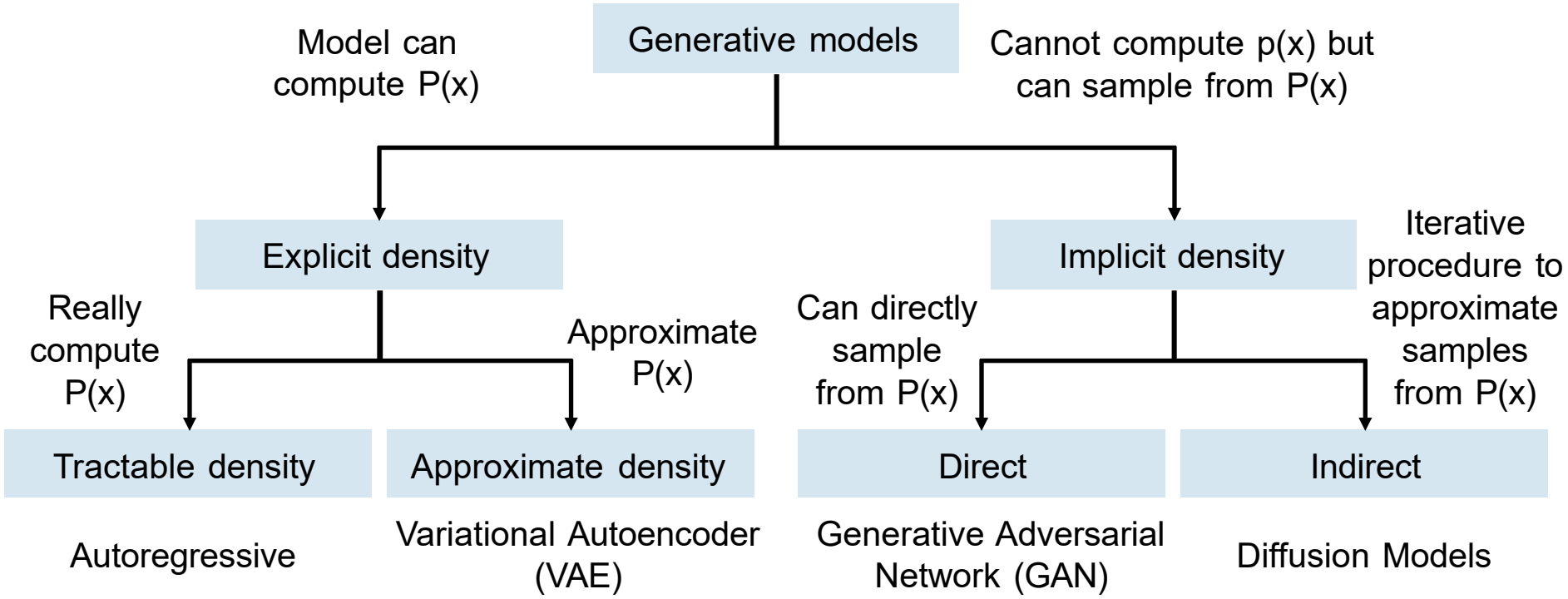
Why Generative Models?

- Modeling ambiguity: If there are many possible outputs x for an input y, we want to model $P(x|y)$
- Language Modeling: Produce output text x from input text y
- Text to Image: Produce output image x from input text y
- Image to Video: What happens next?

Make me an image showing a person teaching a class on generative models in front of a whiteboard



Taxonomy of Generative Models



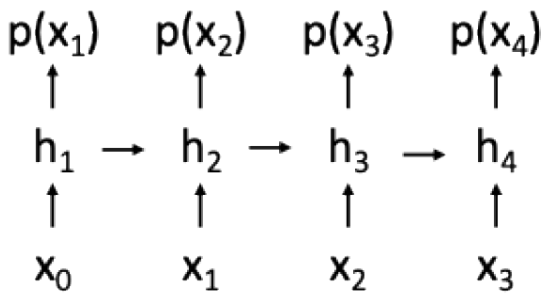
Autoregressive Models

Maximum Likelihood Estimation

- Goal: Write down an explicit function for $p(x) = f(x, W)$
- Given dataset $x^{(1)}, x^{(2)}, \cdots, x^{(N)}$, train the model by solving
- $W^* = argmax_W \prod_i p\left(x^{(i)}\right)$ (Maximize probability of training data)
- $W^* = argmax_W \sum_i \log p\left(x^{(i)}\right)$ (Log trick: Swap product and sum)
- $W^* = argmax_W \sum_i \log f\left(x^{(i)}, W\right)$ (Our loss function, maximize it with gradient descent)

Autoregressive Models

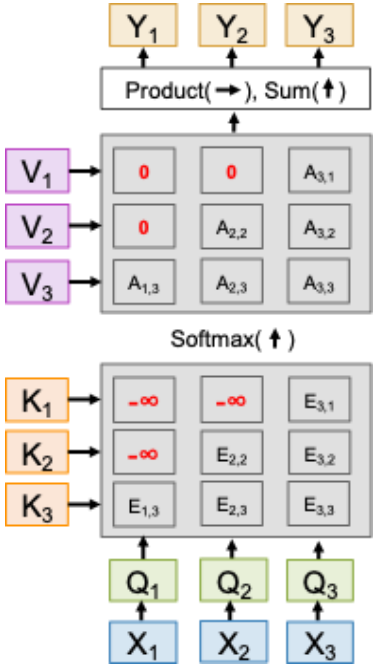
We have already seen this!



Language modeling with RNN

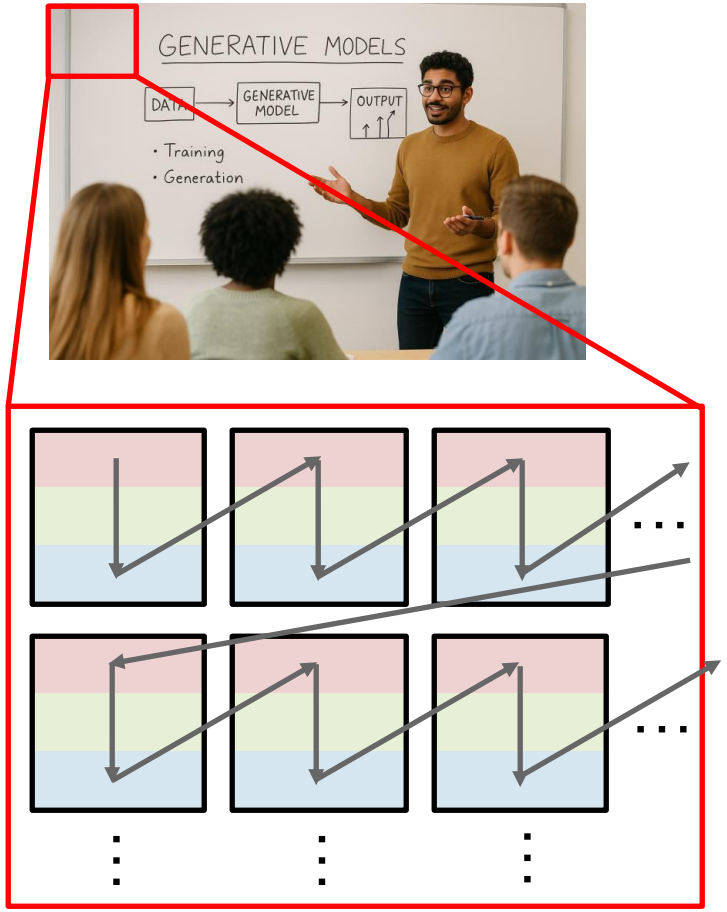
- Goal: Write down an explicit function for $p(x) = f(x, W)$
- Assume x is a sequence: $x = (x_1, x_2, \cdots, x_T)$
- Use the chain rule of probability:
- $p(x) = p(x_1, x_2, \cdots, x_T) = p(x_1)p(x_2, \cdots, x_T | x_1)$
- $p(x) = p(x_1)p(x_2|x_1)p(x_3|x_1, x_2) \cdots$
- $p(x) = \prod_{t=1}^{T} p(x_t | x_1, \cdots, x_{t-1})$

Language modeling with masked Transformer



Autoregressive Models of Images

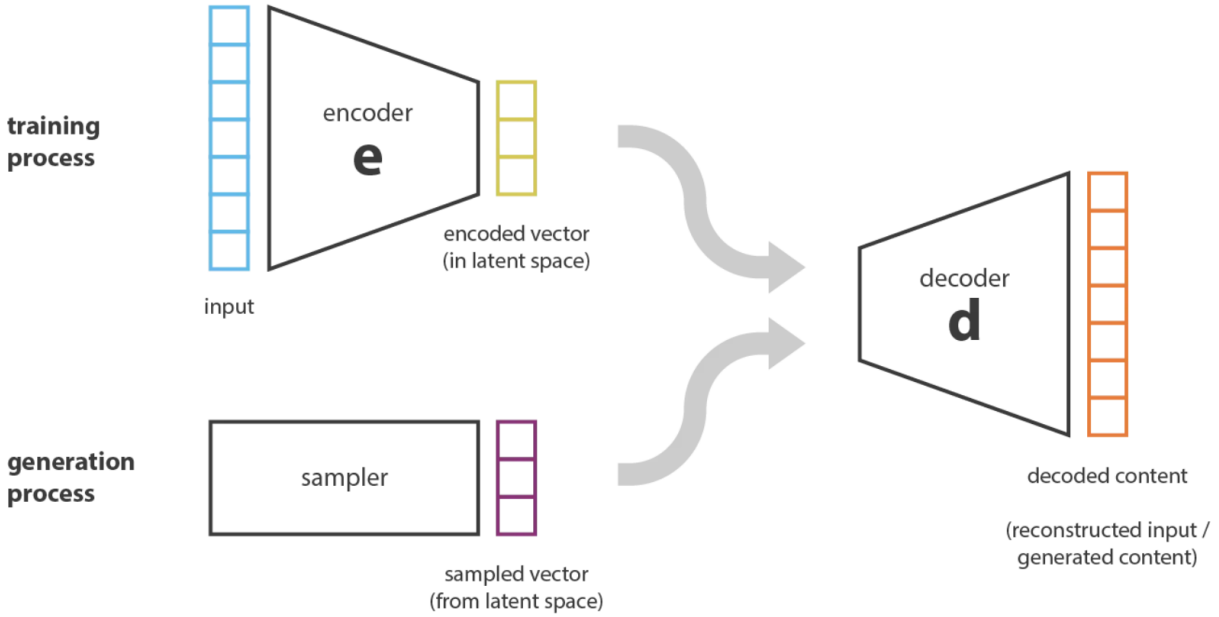
- Treat an image as a sequence of 8-bit subpixel values (scanline order)
- Predict each subpixel as a classification among 256 values [0…255]
- Model with an RNN or Transformer
- **Problem**: Too expensive. 1024x1024 image is a sequence of 3M subpixels
- **Solution** (jumping ahead): Model as sequence of tiles, not sequence of subpixels



From AE to VAE

Limitations of AEs for content generation

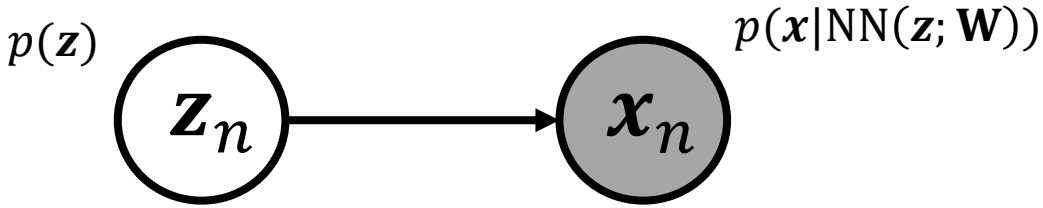
- A trained AE cannot produce any new content.



- Solution: generate new data by decoding points that are randomly sampled from the latent space.

Constructing Generative Models using Neural Nets

- We can use a neural net to define the mapping from a K-dim $\boldsymbol{z}_n$ to D-dim $\boldsymbol{x}_n$



$$\boldsymbol{x}_n \sim \mathcal{N}(\mathrm{NN}(\boldsymbol{z}_n;\mathbf{W}), \sigma^2\mathbf{I}_D)$$

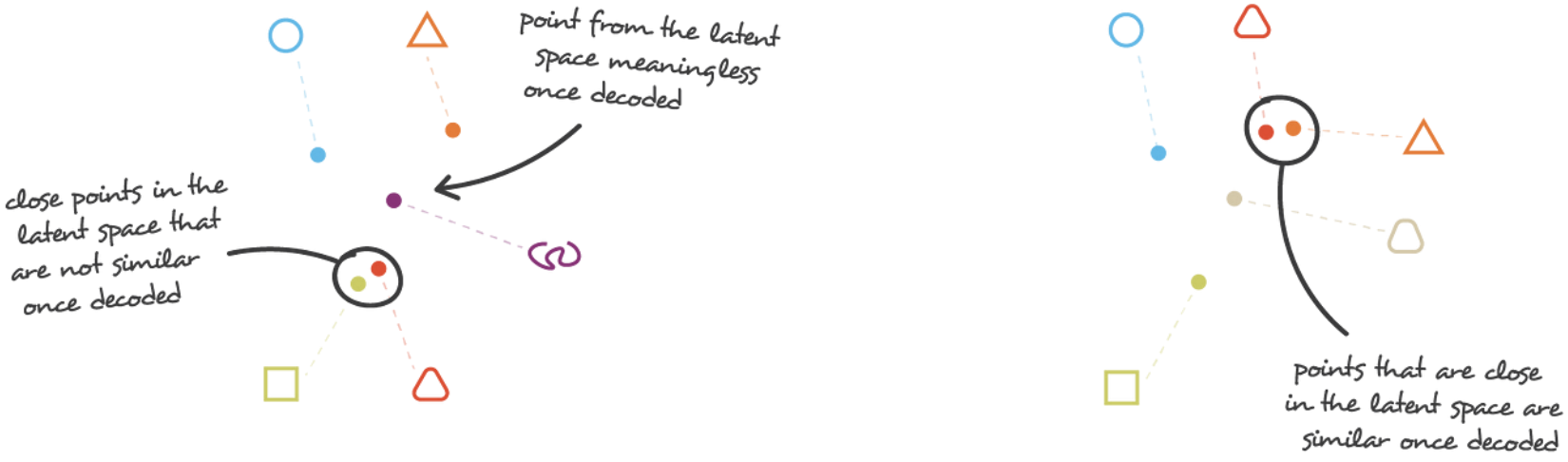
$$\boldsymbol{x}_n \sim \mathrm{ExpFam}(\mathrm{NN}(\boldsymbol{z}_n;\mathbf{W}))$$

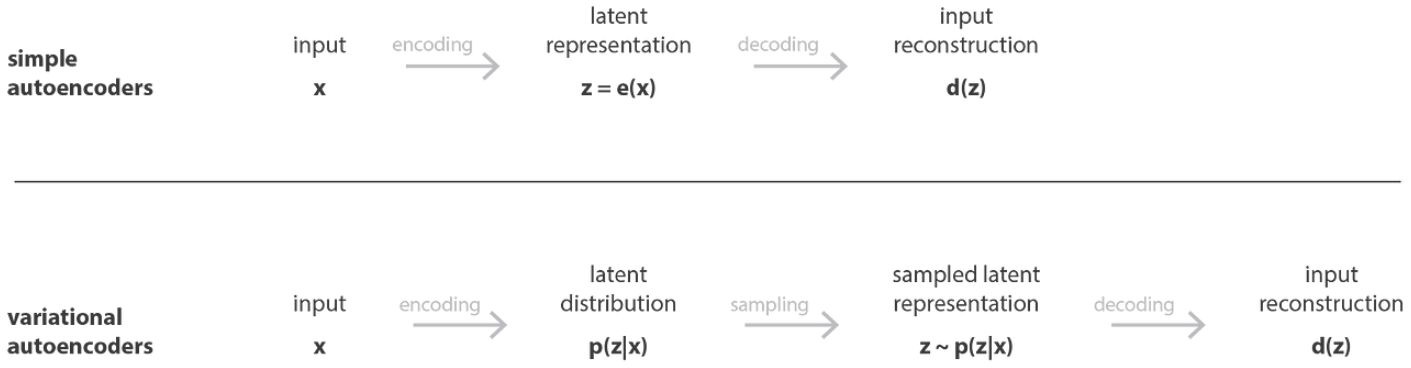
- If $\boldsymbol{z}_n$ has a Gaussian prior, such models are called deep linear Gaussian models (DLGM)
- Since NN mapping is very powerful, DLGM can generate very high-quality data
  - Take the trained network, generate a random $\boldsymbol{z}$ from prior, pass it through the model to generate $\boldsymbol{x}$

Some sample images generated by Vector Quantized Variational AutoEncoder (VQ-VAE), a state-of-the-art DLGM

Variational autoencoder (VAE)

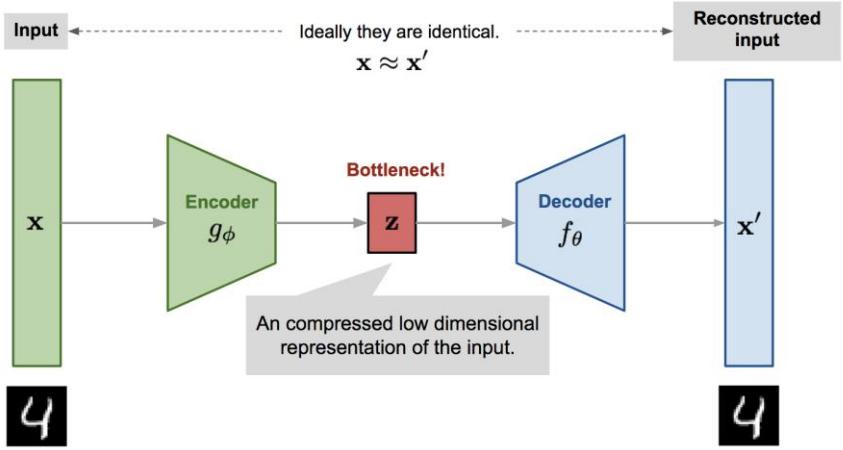
- VAE can be defined as being an AE whose training is regularized to avoid overfitting and ensure that the latent space has good properties that enable generative process.
- Instead of encoding an input as a single point, VAE encodes it as a distribution over the latent space





Variational Autoencoder (VAE)

- VAE* is a probabilistic extension of autoencoders (AE)



$$L_{\mathrm{AE}}(\theta, \phi) = \frac{1}{n} \sum_{i=1}^{n} (\mathbf{x}^{(i)} - f_\theta(g_\phi(\mathbf{x}^{(i)})))^2$$

- The basic difference is that VAE assumes a prior $p(\boldsymbol{z})$ on the latent code $\boldsymbol{z}$
  - This enables it to not just compress the data but also generate synthetic data
  - How: Sample $\boldsymbol{z}$ from a prior and pass it through the decoder
- Thus VAE can learn good latent representation + generate novel synthetic data
- The name has “Variational” in it since it is learned using VI principles

Variational Autoencoders

- PixelRNN / PixelCNN explicitly parameterizes density function with a neural network, so we can train to maximize likelihood of training data:

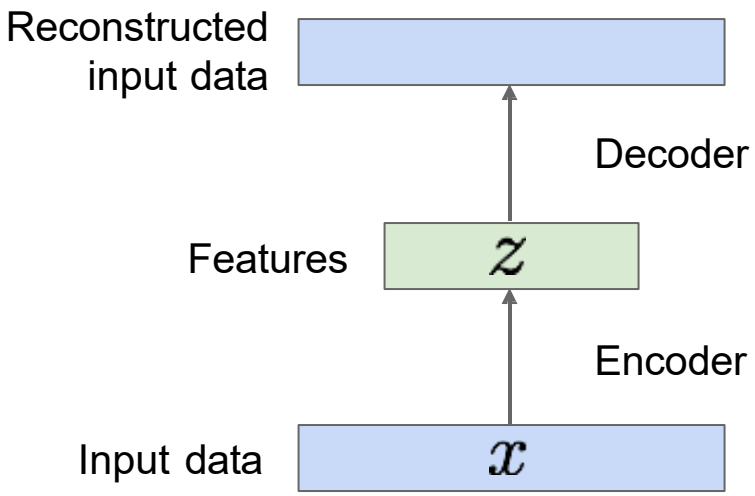
$$p_w(x) = \prod_{t=1}^{T} p_w(x_t | x_1, \cdots, x_{t-1})$$

- Variational Autoencoders (VAE) define an intractable density that we cannot explicitly compute or optimize

- But we will be able to directly optimize a lower bound on the density

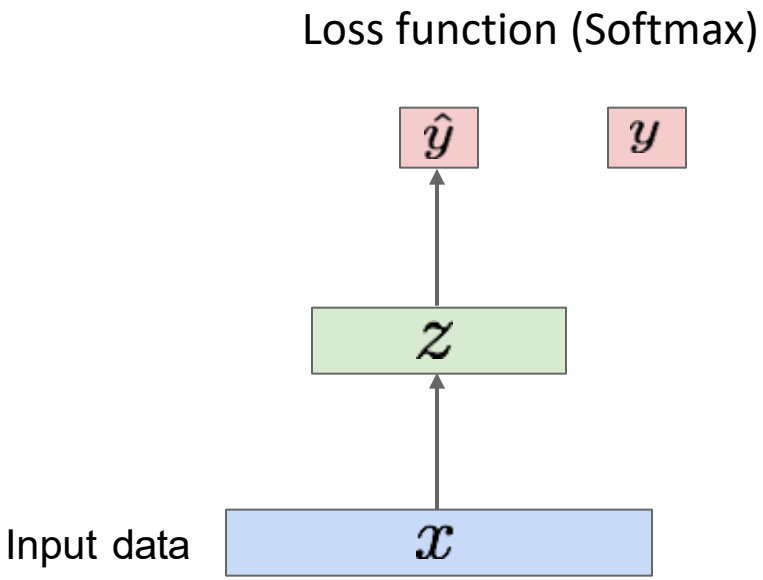
Recap: (Non-Variational) Autoencoders

- Idea: Unsupervised method for learning to extract features z from inputs x, without labels
- Features should extract useful information (object identity, appearance, scene type, etc) that can be used for downstream tasks
- How can we learn without labels?
- Solution: Reconstruct the input data with a decoder.

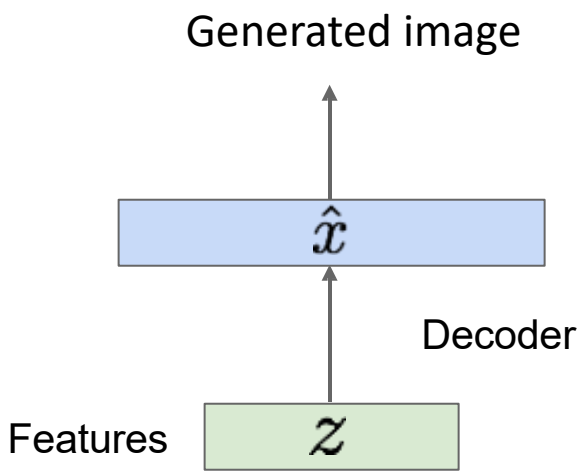


Recap: (Non-Variational) Autoencoders

- After training, can use encoder for downstream tasks



- If we could generate new z, could use the decoder to generate images

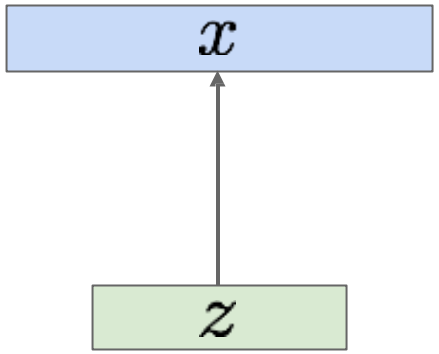


Problem: Generating new z is not any easier than generating new x

Solution: What if we force all z to come from a known distribution?

Motivation: Variational Autoencoders

- Probabilistic spin on autoencoders:
  - 1. Learn latent features z from raw data
  - 2. Sample from the model to generate new data
- Assume training data $\{x^{(i)}\}_{i=1}^{N}$ is generated from unobserved (latent) representation z
- Intuition: x is an image, z is latent factors used to generate x
- After training, sample new data like this:
  - Sample x from conditional $p_{\theta^*}(x|z^{(i)})$
  - Sample z from prior $p_{\theta^*}(z)$
- Assume simple prior $p(z)$ as Gaussian distribution
- How can we train this? Basic Idea: maximum likelihood
  - If we had a dataset of (x,z) then train a conditional generative model $p(x|z)$
  - However, we don't observe z, so marginalize: $p_\theta(x) = \int p_\theta(x,z)dz = \int p_\theta(x|z)p_\theta(z)dz$
  - We can compute $p_\theta(x|z)$ with the **decoder**
  - We assume Gaussian prior for z
  - Problem, we can't integrate over all z



Motivation: Variational Autoencoders

- How can we train this? Basic Idea: maximum likelihood
  - If we had a dataset of (x,z) then train a conditional generative model $p(x|z)$
  - However, we don't observe z, so marginalize: $p_\theta(x) = \int p_\theta(x,z)dz = \int p_\theta(x|z)p_\theta(z)dz$
  - We can compute $p_\theta(x|z)$ with the **decoder**
  - We assume Gaussian prior for z
  - Problem, we can't integrate over all z
- Another idea: Try Bayes' rule:
  - $p(x|y) = \frac{p(y|x)}{p(y)}p(x)$
  - $p_\theta(x) = \frac{\boldsymbol{p_\theta(x|z)}p_\theta(z)}{p_\theta(z|x)}$, why?
  - We can compute $\boldsymbol{p_\theta(x|z)}$ with the decoder
  - We can assume Gaussian prior for $p_\theta(z)$
  - Problem, no way to compute $p_\theta(z|x)$
  - Solution: Train another network that learns $q_\phi(z|x) \approx p_\theta(z|x)$

$$p_\theta(x) \approx \frac{\boldsymbol{p_\theta(x|z)}p_\theta(z)}{q_\phi(z|x)}$$

ELOB and VAE

Variational Autoencoders

- Decoder Network:
  - Input latent code z,
  - Output distribution over data x

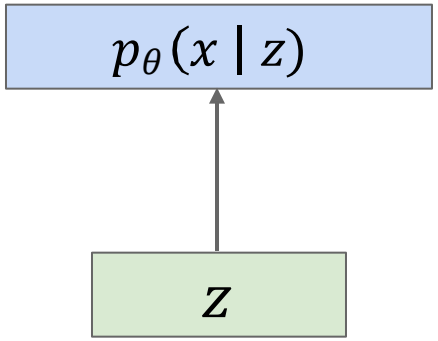
- Encoder Network:
  - Input data z,
  - Output distribution over latent codes z

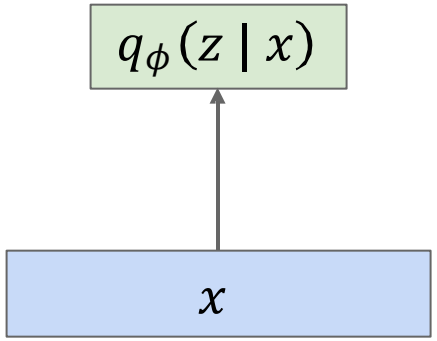
- If we can ensure that

$$q_\phi(z|x) \approx p_\theta(z|x)$$

- Then we can approximate

$$p_\theta(x) \approx \frac{\boldsymbol{p_\theta(x|z)}p_\theta(z)}{q_\phi(z|x)}$$





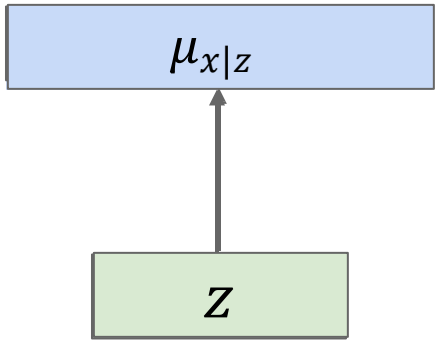
- Idea: Jointly train both encoder and decoder
- Q: How to output probability distributions from neural networks
- A: Network outputs mean (and std) of a (diagonal) distribution

Variational Autoencoders

- Decoder Network:
  - Input latent code z,
  - Output distribution over data x

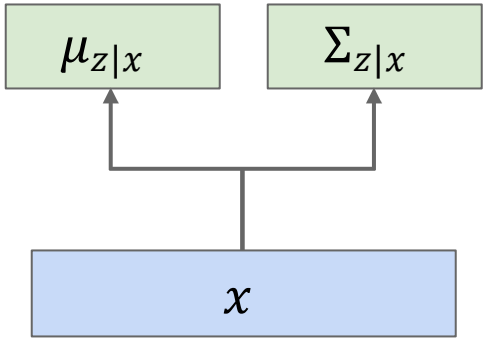
$$p_\theta(x|z) = N(\mu_{x|z}, \sigma^2)$$

$$\log p_\theta(x|z) = -\frac{1}{2\sigma^2}||x-\mu||_2^2 + C_2$$



- Encoder Network:
  - Input data z,
  - Output distribution over latent codes z

$$q_\phi(z|x) = N(\mu_{x|z}, \Sigma_{z|x})$$



- If we can ensure that

$$q_\phi(z|x) \approx p_\theta(z|x)$$

- Then we can approximate

$$p_\theta(x) \approx \frac{\boldsymbol{p_\theta(x|z)}p_\theta(z)}{q_\phi(z|x)}$$

- Idea: Jointly train both encoder and decoder
- Q: How to output probability distributions from neural networks
- A: Network outputs mean (and std) of a (diagonal) distribution
- Maximizing $\log p_\theta(x|z)$ is equivalent to minimizing L2 distance between x and network output!
- What is the training objective?

VAE (Evidence Lower Bound, ELOB)

- $\log p_\theta(x) = \log \frac{p_\theta(x|z)p(z)}{p_\theta(z|x)}$ (Bayes' rule)
- $\log p_\theta(x) = \log \frac{p_\theta(x|z)p(z)q_\phi(z|x)}{p_\theta(z|x)q_\phi(z|x)}$ (Multiply top and bottom by $q_\phi(z|x)$)
- $\log p_\theta(x) = \log p_\theta(x|z) - \log \frac{q_\phi(z|x)}{p(z)} + \log \frac{q_\phi(z|x)}{p_\theta(z|x)}$
- $\log p_\theta(x) = E_z[p_\theta(x|z)] - E_z\left[\frac{q_\phi(z|x)}{p(z)}\right] + E_z\left[\log \frac{q_\phi(z|x)}{p_\theta(z|x)}\right]$
- $\log p_\theta(x) = E_{z\sim q_\phi(z|x)}[p_\theta(x|z)] - E_z\left[\frac{q_\phi(z|x)}{p(z)}\right] + E_z\left[\log \frac{q_\phi(z|x)}{p_\theta(z|x)}\right]$
- $\log p_\theta(x) = E_{z\sim q_\phi(z|x)}[p_\theta(x|z)] - D_{KL}\left(q_\phi(z|x), p(z)\right) + D_{KL}\left(q_\phi(z|x), p_\theta(z|x)\right)$
- $E_{z\sim q_\phi(z|x)}[p_\theta(x|z)]$: Data reconstruction: x=>encoder=>decoder should reconstruct x. Can compute in closed form for Gaussians
- $D_{KL}\left(q_\phi(z|x), p(z)\right)$: Prior: encoder output should match prior over z. Can compute in closed for for Gaussians
- $D_{KL}\left(q_\phi(z|x), p_\theta(z|x)\right)$: Posterior Approximation: encoder output should match $p_\theta(z|x)$. Cannot compute.

- Why $E_z[p_\theta(x|z)] = E_{z\sim q_\phi(z|x)}[p_\theta(x|z)]$?
- Expectation in Math:
  $$E_{z\sim q(z)}[c] = \int cq(z)dz$$
- If c is a constant (irrelevant to z), then
  $$\int cq(z)dz = c\int q(z)dz = c \cdot 1 = c$$
- Then
  $$E_z[p_\theta(x|z)] = E_{z\sim q_\phi(z|x)}[p_\theta(x|z)]$$
  $$= \log p_\theta(x)$$

VAE (Evidence Lower Bound, ELOB)

- $\log p_\theta(x) = E_{z\sim q_\phi(z|x)}[p_\theta(x|z)] - D_{KL}\left(q_\phi(z|x), p(z)\right) + D_{KL}\left(q_\phi(z|x), p_\theta(z|x)\right)$
- $E_{z\sim q_\phi(z|x)}[p_\theta(x|z)]$: Data reconstruction: x=>encoder=>decoder should reconstruct x. Can compute in closed form for Gaussians
- $D_{KL}\left(q_\phi(z|x), p(z)\right)$: Prior: encoder output should match prior over z. Can compute in closed for for Gaussians
- $D_{KL}\left(q_\phi(z|x), p_\theta(z|x)\right)$: Posterior Approximation: encoder output should match $p_\theta(z|x)$. Cannot compute for Gaussian.
- KL divergence ≥ 0. So we can drop it to get a lower bound on likelihood
- $\log p_\theta(x) \geq E_{z\sim q_\phi(z|x)}[p_\theta(x|z)] - D_{KL}\left(q_\phi(z|x), p(z)\right)$, this is our VAE training objective.

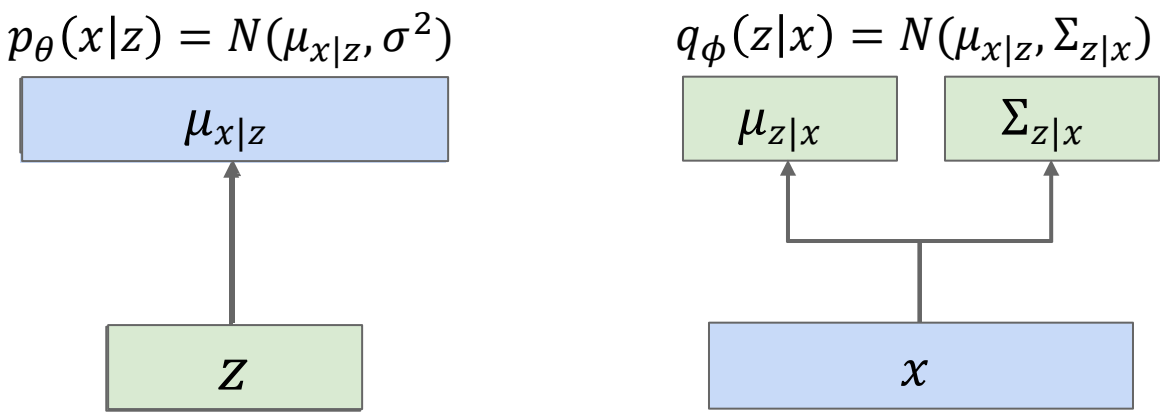
Closed form calculation

- $E_{z\sim q_\phi(z|x)}[p_\theta(x|z)]$: Data reconstruction: x=>encoder=>decoder should reconstruct x. Can compute in closed form for Gaussians
- $\log p_\theta(x|z) = -\frac{1}{2\sigma^2}\left|\left|\mathrm{x} - \mu_\theta(z)\right|\right|^2 - \frac{D}{2}\log(2\pi\sigma^2)$
- $E_{z\sim q_\phi(z|x)}[p_\theta(x|z)] = -\frac{1}{2\sigma^2}E_{q(z|x)}\left|\left|\mathrm{x} - \mu_\theta(z)\right|\right|^2 + const$

- $D_{KL}\left(q_\phi(z|x), p(z)\right)$: Prior: encoder output should match prior over z. Can compute in closed for for Gaussians
- For two Gaussian distribution $q = N(\mu_q, \Sigma_q)$ and $p = N(\mu_p, \Sigma_p)$
- KL divergence: $KL(p,q) = \frac{1}{2}\left(\log\frac{|\Sigma_p|}{|\Sigma_q|} - k + tr\left(\Sigma_p^{-1}\Sigma_q\right) + \left(\mu_p - \mu_q\right)^T\Sigma_p^{-1}\left(\mu_p - \mu_q\right)\right)$
- For VAE, $\mu_p = 0, \Sigma_p = I, \Sigma_q = diag(\sigma^2), KL = \frac{1}{2}\sum_i^k\left(\sigma_i^2 + \mu_i^2 - 1 - \log\sigma_i^2\right)$

VAE (ELOB)

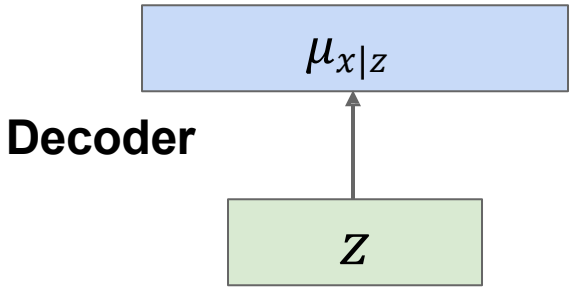
- Jointly train encoder q and decoder p to maximize the variational lower bound on the data likelihood
- Also called Evidence Lower Bound(ELOB)
- $\log p_\theta(x) \geq E_{z\sim q_\phi(z|x)}[p_\theta(x|z)] - D_{KL}\left(q_\phi(z|x), p(z)\right)$



VAE Training

- Train by maximizing the variational lower bound
- $\log p_\theta(x) \geq E_{z\sim q_\phi(z|x)}[p_\theta(x|z)] - D_{KL}\left(q_\phi(z|x), p(z)\right)$
- 1. Run input data through encoder to get distribution over z
- 2. Prior loss: Encoder output should be unit Gaussian (zero mean, unit variance)
- 3. Sample z from encoder output $q_\phi(z|x)$ (Reparameterization trick)
- 4. Run z through decoder to get predicted data mean
- 5. Reconstruction loss: predicted mean should match x in L2

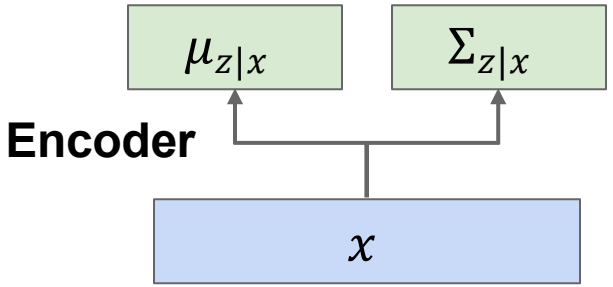
$$p_\theta(x \mid z) = N(\mu_{x|z}, \sigma^2)$$



$$\epsilon \sim N(0, I)$$

$$z = \epsilon \odot \Sigma_{z|x} + \mu_{z|x}$$

$$q_\phi(z \mid x) = N(\mu_{z|x}, \Sigma_{z|x})$$

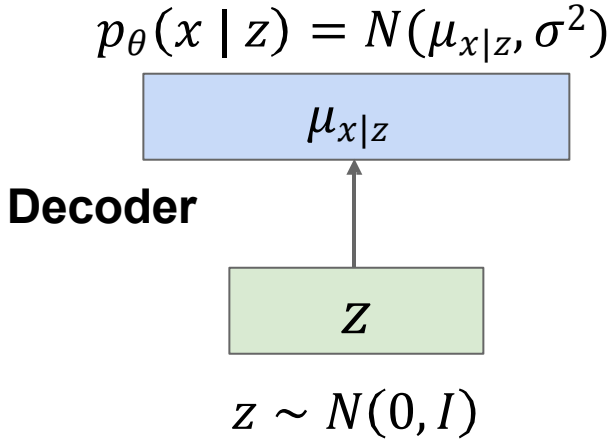


Reparameterization trick

- **Problem**: VAE optimizes the ELOB $E_{z\sim q_\phi(z|x)}[p_\theta(x|z)] - D_{KL}\left(q_\phi(z|x), p(z)\right)$
  - The latent variable z is sampled from $q_\phi(z|x)$
  - Direct sampling introduces a **non-differentiable stochastic node**
  - Gradient cannot propagate back to encoder parameters ϕ
- **Key idea**: Separate **randomness** from **learnable parameters**
  - Rewrite sampling as a deterministic transformation of noise
  - $z = \mu\phi(x) + \sigma\phi(x) \odot \varepsilon, \varepsilon \sim N(0, I)$
- **Why It Works**
  - Randomness comes only from ε, which is independent of ϕ
  - $\mu_\phi(x)$and $\sigma_\phi(x)$are deterministic and differentiable
  - Enables **end-to-end backpropagation** through the encoder

VAE Sampling

- Sample z from the prior
- Run through decoder to get an image



Summary of VAE

- VAE has three main components
  - A prior $p_\theta(\boldsymbol{z})$ over latent codes
  - A probabilistic decoder $p_\theta(\boldsymbol{x}|\boldsymbol{z})$
  - A posterior or probabilistic encoder $p_\theta(\boldsymbol{z}|\boldsymbol{x})$ approx. by an "inference network" $q_\phi(\boldsymbol{z}|\boldsymbol{x})$
- VAE is learned by maximizing the ELBO

Here θ collectively denotes all the parameters of the prior and likelihood

Here ϕ collectively denotes all the parameters that define the inference network

ELBO for a single data point

$$\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\phi}|\mathbf{x}) = \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\left[\log p_{\boldsymbol{\theta}}(\mathbf{x}, \mathbf{z}) - \log q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})\right]$$

$$= \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\left[\log p_{\boldsymbol{\theta}}(\mathbf{x}|\mathbf{z})\right] - \mathbb{KL}\left(q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x}) \| p_{\boldsymbol{\theta}}(\mathbf{z})\right)$$

Maximized to find the optimal θ and ϕ

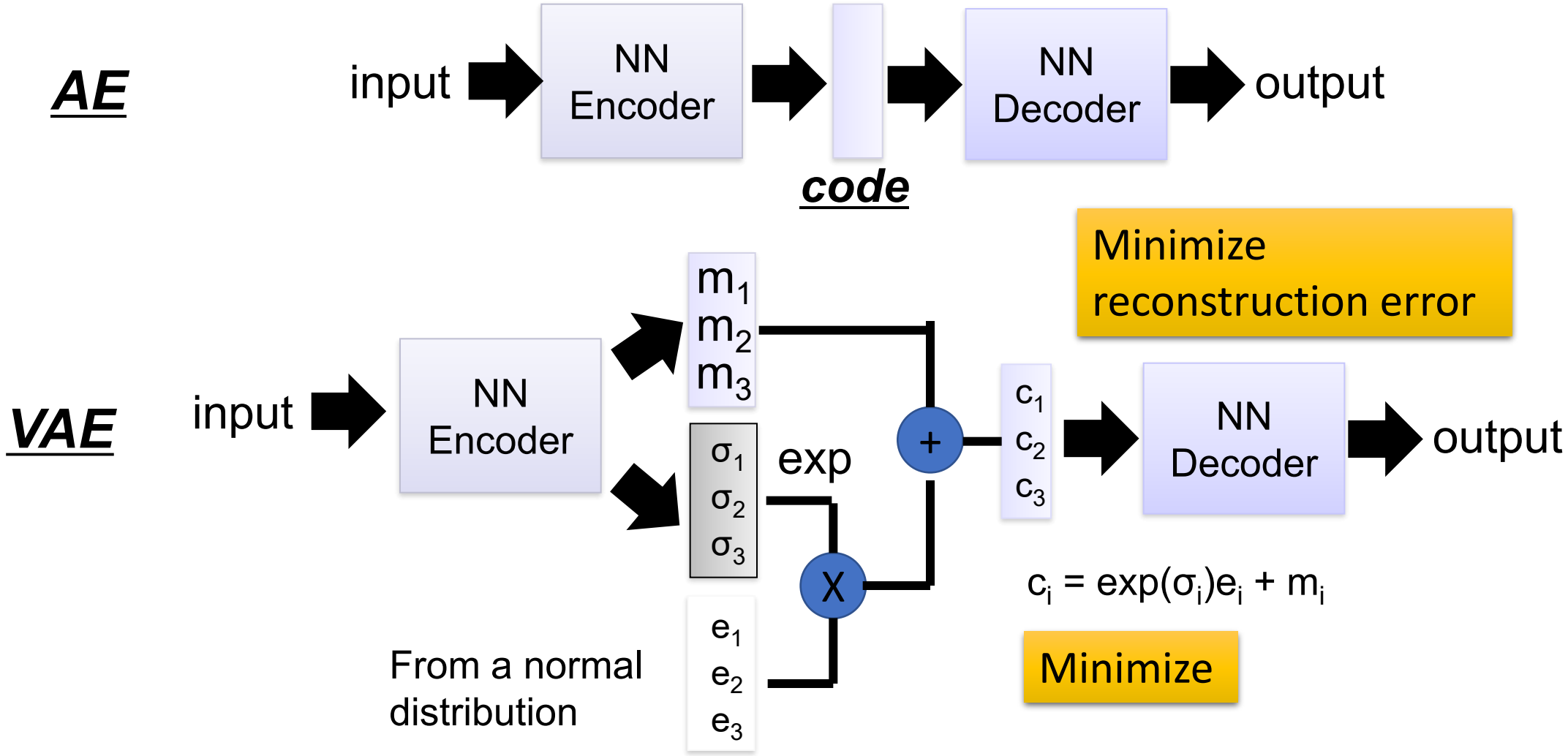
q_ϕ should reconstruct the data x well from z(high log-like)

q_ϕ should also be simple (close to the prior)

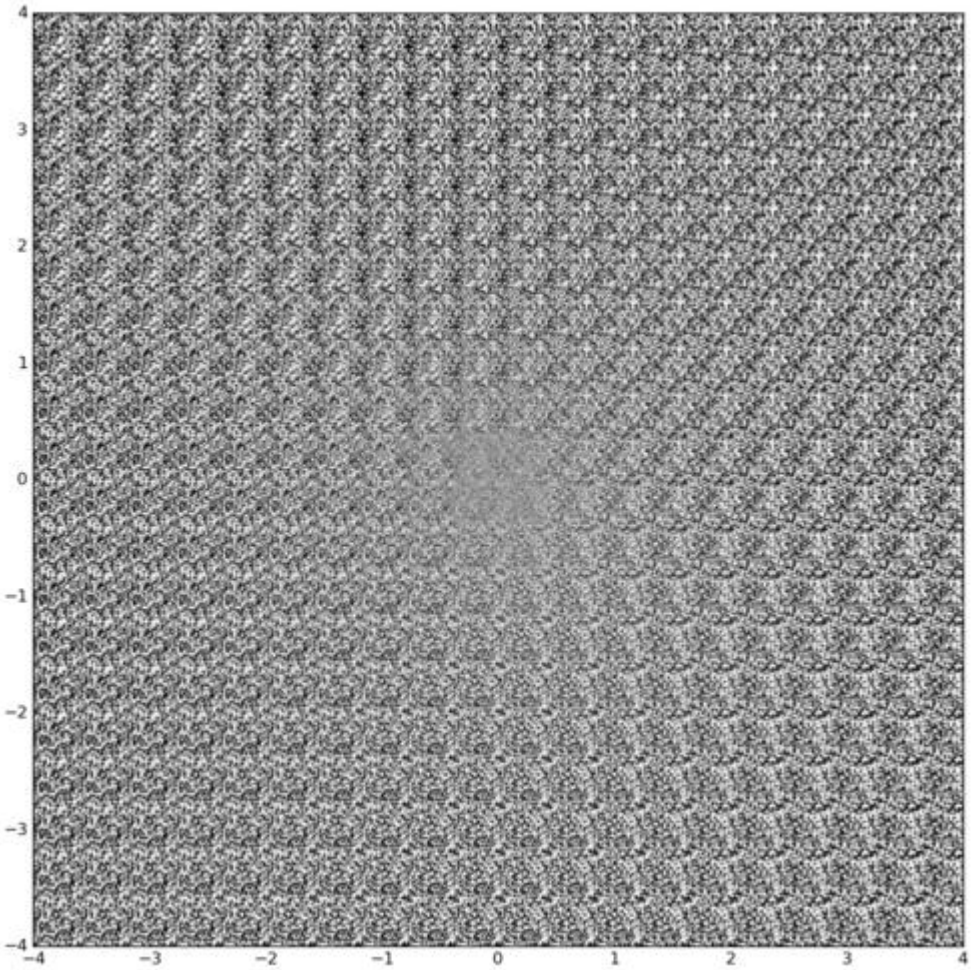
- The Reparametrization Trick is commonly used to optimize the ELBO
- Posterior is inferred only over z, and usually only point estimate on θ and ϕ

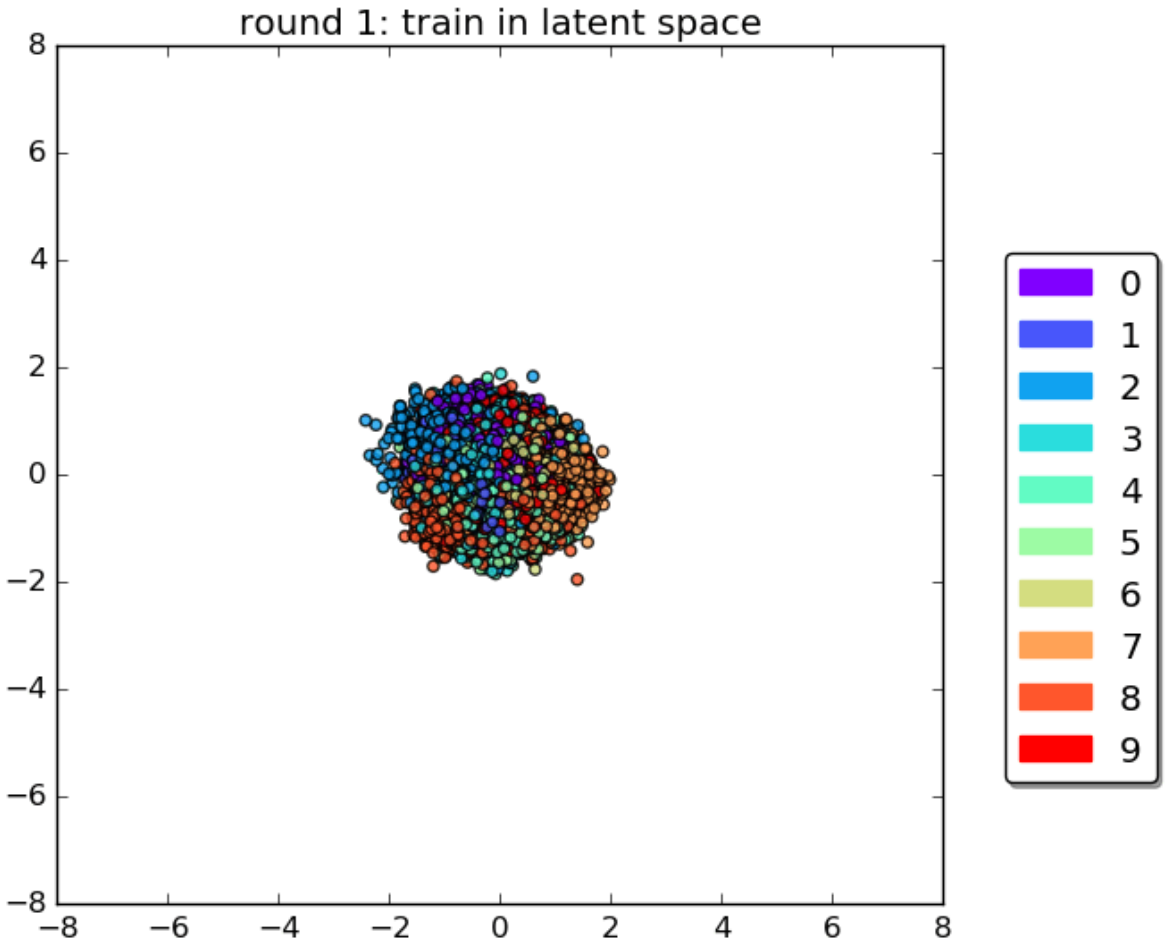
Applications

AE vs VAE

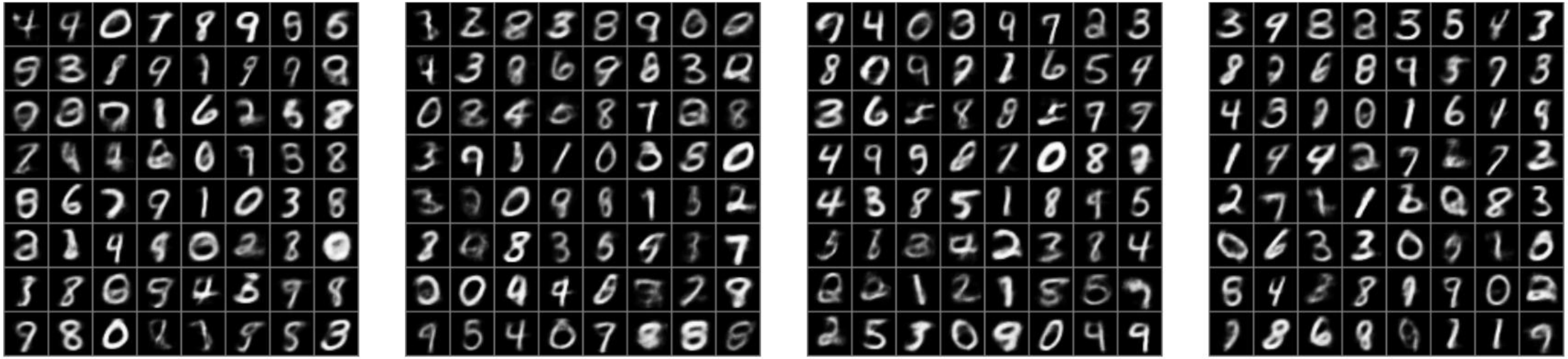

AE
input
NN Encoder
code
NN Decoder
output
Minimize reconstruction error
VAE
input
NN Encoder
m_1 m_2 m_3
σ_1 σ_2 σ_3
exp
+
X
e_1 e_2 e_3
From a normal distribution
c_1 c_2 c_3
NN Decoder
output
$c_i = \exp(\sigma_i)e_i + m_i$
Minimize

Generating new images on MNIST



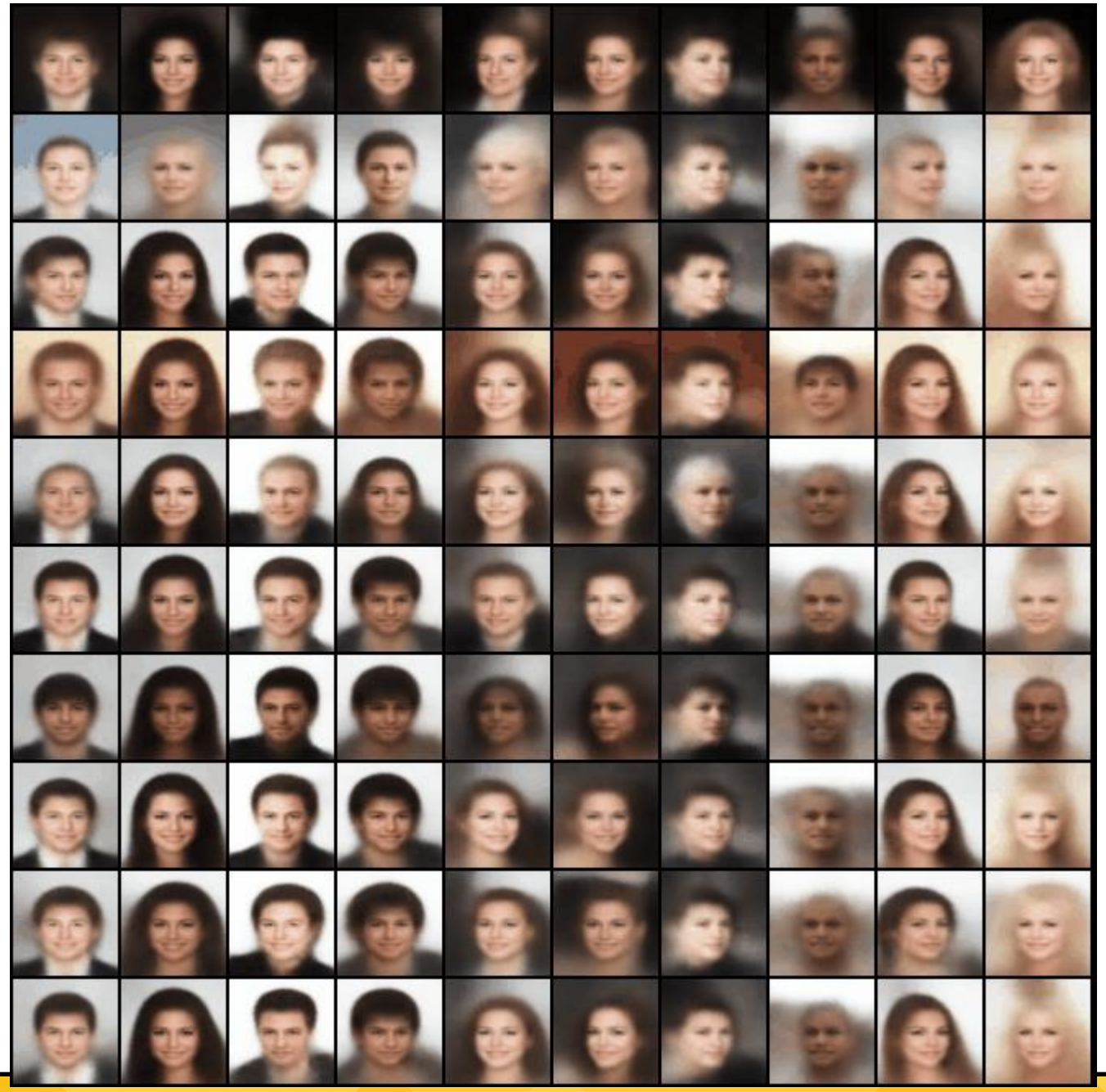


Generating new images on MNIST



2.5 D face generation

Face editing



https://medium.com/analytics-vidhya/generative-modelling-using-variational-autoencoders-vae-and-beta-vaes-81a56ef0bc9f

Summary

- Introduction
- Autoregressive Models
- From AE to VAE
- ELOB and VAE
- Application of VAE